HILOS
ASPECTOS PRACTICOS

Tipos de datos

Se tratan como datos "opacos"

No se puede comparar dos identificadores de hilos usando el operador "=="

Se debe usar funciones específicas de la API

Data type	Description
pthread_t	Thread identifier
pthread_mutex_t	Mutex
pthread_mutexattr_t	Mutex attributes object
pthread_cond_t	Condition variable
pthread_condattr_t	Condition variable attributes object
pthread_key_t	Key for thread-specific data
pthread_once_t	One-time initialization control context
pthread_attr_t	Thread attributes object

Manejo de *errno* por hilo

errno nos indica el error que ocurrio

Genera un problema de sobre escritura entre dos hilos

La solucion: cada hilo tiene su propia copia privada de errno

Requisito: agregar <errno.h>

```c
#include <stdio.h>

#include <errno.h>

#include <string.h>


int main() {

   FILE *f = fopen("archivo_inexistente.txt", "r");

   if (f == NULL) {

      printf("Error al abrir el archivo: %s¥n", strerror(errno));

   }

   return 0;

}
```

Valores de retorno

En funciones tradicionales:

- 0 en caso de éxito
- -1 en cado de error

En funciones de Pthreads:

- 0 en caso de éxito
- Numero positivo en caso de error

Creacion de hilo

```
#include <pthread.h>

int pthread_create(pthread_t *thread, const pthread_attr_t *attr,
                   void *(*start)(void *), void *arg);
```

Returns 0 on success, or a positive error number on error

- Pthread_t *thread = ID
- const pthread_attr_t *attr = Atributos
- void *(*start)(void *) = Funcion de inicio
- void *arg = Argumento

Terminación de hilos

Las 4 formas en que muere un hilo:

- Return
- Pthread_exit()
- Pthread_cancel()
- exit()

Identificadores de hilo

¿Que es un ID de un hilo?

- Pthread_create()
- Pthread_self()

```
include <pthread.h>

pthread_t pthread_self(void);
```

¿Para que sirve?

- Identificacion en funciones
- Etiquetado de datos
- Toma de decision

```
if (pthread_equal(tid, pthread_self())
    printf("tid matches self\n");
```

Comparacion de IDs

No se utiliza el operador "==" para comparar dos IDs

Devuelve valor distinto de 0 si son iguales

Union de hilos

La función pthread_join() permite que un hilo espere a que otro hilo específico termine su ejecución y limpiar memoria del sistema

```
s = pthread_create(&t1, NULL, threadFunc, "Hello world\n");
if (s != 0)
    errExitEN(s, "pthread_create");

printf("Message from main()\n");
s = pthread_join(t1, &res);
if (s != 0)
    errExitEN(s, "pthread_join");

printf("Thread returned %ld\n", (long) res);

exit(EXIT_SUCCESS);
```

Desacoplamiento de hilos

Caracteristica	Unir	Desacoplar
Esperas al hilo	Si, hasta que termine	No, sigue corriendo
Recibis resultado	Si, obtenes valor de retorno	No, se pierde el resultado
Quien limpia la memoria	Al hacer la llamada join	El sistema automaticamente
Estado por defecto	Estado normal del hilo	Agregarlo explicitamente

Atributos de hilos

```
pthread_t thr;
pthread_attr_t attr;
int s;

s = pthread_attr_init(&attr);                /* Assigns default values */
if (s != 0)
    errExitEN(s, "pthread_attr_init");

s = pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
if (s != 0)
    errExitEN(s, "pthread_attr_setdetachstate");

s = pthread_create(&thr, &attr, threadFunc, (void *) 1);
if (s != 0)
    errExitEN(s, "pthread_create");

s = pthread_attr_destroy(&attr);             /* No longer needed */
if (s != 0)
    errExitEN(s, "pthread_attr_destroy");
```

Inicializa la estructura con los valores por defecto

Cambia los atributos para que el hilo sea desacoplado

Se crea el hilo usando esa configuracion

Cuando se crea el hilo, el objeto de atributo ya no es necesario y se destruye para liberar espacio

Hilos vs procesos

Ventajas de los hilos

1. Transferencia de datos
2. Velocidad

Desventajas

1. Falta de aislamiento
2. Hilos-seguros
3. Competencia por espacio
4. Manejo de señales

GRACIAS